

Java Backend Architecture Interview Series

Sub-Chapter 1 1

Java Primitives & Type System

The 8 Types | Wrappers | Autoboxing | Integer Cache | IEEE 754 | String Pool | equals/hashCode

40+ pages & 28 Q&A | Standard + Advanced Interview Questions | Working Code Examples

1 The 8 Primitive Types

1.1 Introduction & Motivation

Java's type system has two distinct tiers: primitive types and reference types. Primitives are the low-level building blocks: raw bit patterns stored directly on the stack or inline in object fields, with no object header overhead, no inheritance, and no null. Understanding primitives deeply is foundational for every Java interview because they underpin autoboxing pitfalls, Integer cache surprises, IEEE 754 floating-point traps, and the equals/hashCode contract. This section provides the complete reference table and explains every nuance you will encounter in production code.

1.2 Complete Primitives Reference Table

Type	Bits	Min Value	Max Value	Wrapper	Default	Suffix
byte	8	-128	127	Byte	0	-
short	16	-32,768	32,767	Short	0	-
int	32	-2,147,483,648	2,147,483,647	Integer	0	-
long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	Long	0L	L
float	32	~1.4e-45	~3.4e+38 (7 sig dig)	Float	0.0f	F
double	64	~4.9e-324	~1.8e+308 (15 sig dig)	Double	0.0d	D
char	16	0 (\u0000)	65,535 (\uFFFF)	Character	\u0000	-
boolean	1*	false	true	Boolean	false	-

* boolean size is JVM-implementation-defined; in arrays the JVM typically allocates 1 byte per element.

1.3 Numeric Literals & Underscore Syntax (Java 7+)

Java 7 introduced underscore separators in numeric literals, making large numbers dramatically more readable. Underscores can appear between any two digits but NOT at the start, end, or adjacent to a decimal point or suffix.

```
// No imports needed all primitive literal syntax is built-in

// Decimal, Hex, Octal, Binary literals

int dec = 1_000_000; // underscore separator (Java 7+)

int hex = 0xFF_EC_D1_B4; // hexadecimal 0x prefix

int oct = 0777; // octal leading 0 (rarely used)

int bin = 0b1010_1010; // binary 0b prefix (Java 7+)

long big = 9_223_372_036L; // L suffix required for long literal

// Floating-point literals

float f1 = 3.14f; // f/F suffix required else double

double d1 = 3.14; // default floating-point type is double
```

```
double d2 = 6.02e23; // scientific notation

// char literals
char c1 = 'A'; // single character
char c2 = '\u0041'; // Unicode escape also 'A'
char c3 = 65; // integer value in [0, 65535]
char c4 = '\n'; // escape sequences: \t \r \\ \n
```

Numeric and char literals all forms

1.4 Default Values: Fields vs Local Variables

WARNING: Local variables **MUST** be explicitly initialised before use the compiler reports a compile error. Only instance and static (class) fields receive automatic default values. This is a common source of confusion for newcomers.

```
class DefaultDemo {

    int instanceInt; // default 0

    double instanceDb1; // default 0.0

    boolean instanceBool; // default false

    String instanceStr; // default null (reference type)

    void method() {

        int localInt; // NO default value

        // System.out.println(localInt); // compile error!

        localInt = 5; // explicit init required

        System.out.println(localInt); // OK

    }

}
```

Field defaults vs local variable requirement

2 Type Widening, Narrowing & Numeric Promotion

2.1 The Problem Implicit and Explicit Conversions

Java allows implicit (widening) conversions when no data is lost, and requires explicit casts for narrowing conversions where data may be truncated. The promotion rules define the result type of arithmetic so expressions always have a well-defined type. Misunderstanding these rules causes subtle compile errors and silent data loss.

2.2 Widening (Implicit) Conversions

The widening hierarchy: byte short int long float double. char widens to int as an unsigned 16-bit value. WARNING: long to float and long to double can silently lose precision because float has only ~7 significant decimal digits.

```
// Safe widening no cast required

byte b = 42;

short s = b; // byte short

int i = s; // short int

long l = i; // int long

float f = l; // long float (may lose precision!)

double d = f; // float double


// PRECISION LOSS even though it is widening!

long bigLong = 123_456_789_012_345L;

float asFloat = bigLong; // 1.2345679E14 digits lost!

double asDouble = bigLong; // 1.2345678901234E14 some loss


// char is unsigned 16-bit; widens to int with no sign extension

char ch = 'A'; // 65

int n = ch; // 65 no cast needed
```

Widening conversions hierarchy and precision pitfall

2.3 Narrowing (Explicit Cast) Truncation & Bit Masking

```
// Narrowing requires explicit cast data may be lost

int bigInt = 300;

byte narrow = (byte) bigInt; // 300 & 0xFF = 44 (bit truncation)

// 300 = 0b00000001_00101100; keep low 8 bits: 0b00101100 = 44


double pi = 3.14159;
```

```
int intPi = (int) pi; // 3 truncation toward zero, NOT rounding!

int neg = (int) -3.99; // -3 also truncation toward zero

// char <-> int narrowing/widening

int code = 65;

char cc = (char) code; // 'A'

int back = cc; // 65 (widening no cast)

// Classic trap: byte arithmetic result is int!

byte x = 10, y = 20;

// byte z = x + y; // COMPILER ERROR: int cannot assign to byte

byte z = (byte)(x + y); // OK explicit narrowing cast
```

Narrowing conversions casting rules and traps

2.4 Numeric Promotion Rules

Operation	Rule	Example
byte/short/char + any	Both operands promoted to int first	byte a=10; byte b=20; int c=a+b;
int + long	int promoted to long	int i=5; long l=10L; long r=i+l;
int/long + float	Integral promoted to float	long l=9L; float r=l+3.14f;
any + double	Other operand promoted to double	int i=3; double r=i+1.5;
Compound ops (+=, *=)	Implicit narrowing cast included	byte b=10; b+=5; // OK

TIP: Compound assignment operators (+=, -=, *=, /=) include an implicit narrowing cast. This is why 'byte b=10; b+=5;' compiles but 'byte b=10; b=b+5;' does not the plain assignment requires an explicit (byte) cast.

3 Integer Overflow & IEEE 754 Floating Point

3.1 The Problem Silent Data Corruption

Integer arithmetic in Java silently wraps on overflow: no exception, no warning. Floating-point arithmetic follows IEEE 754, which introduces special values (Infinity, NaN) and cannot represent most decimal fractions exactly. Both behaviours cause production bugs when developers assume arithmetic 'just works like math'. This section explains the exact rules and the safe alternatives.

3.2 Integer Overflow Silent Wrap-Around

WARNING: Java integer arithmetic **SILENTLY** wraps on overflow no `ArithmeticException`, no flag. `Integer.MAX_VALUE + 1 = Integer.MIN_VALUE`. Use `Math.addExact()` / `Math.multiplyExact()` for checked arithmetic (throws `ArithmeticException` on overflow).

```
import java.lang.Math; // Math.addExact, Math.multiplyExact, Math.absExact

// Silent integer overflow no exception!

int maxInt = Integer.MAX_VALUE; // 2_147_483_647

int wrapped = maxInt + 1; // -2_147_483_648 (MIN_VALUE!)

long maxLong = Long.MAX_VALUE;

long wrappedL = maxLong + 1; // Long.MIN_VALUE

// Safe checked arithmetic (Java 8+) throws ArithmeticException

int safe = Math.addExact(maxInt, 1); // throws!

long mul = Math.multiplyExact(100_000L, 100_000L); // OK: 10_000_000_000

// Negating MIN_VALUE overflows back to MIN_VALUE!

System.out.println(-Integer.MIN_VALUE == Integer.MIN_VALUE); // true!

// Math.absExact(Integer.MIN_VALUE) throws ArithmeticException (Java 15+)
```

Integer overflow silent wrap and safe alternatives

3.3 IEEE 754 Special Values Infinity and NaN

```
// All special values are constants on java.lang.Double (auto-imported)

// Float/double division by zero returns special values (no exception)

double posInf = 1.0 / 0.0; // Double.POSITIVE_INFINITY

double negInf = -1.0 / 0.0; // Double.NEGATIVE_INFINITY

double nan = 0.0 / 0.0; // Double.NaN (Not a Number)

// NaN is NOT equal to itself IEEE 754 rule

System.out.println(nan == nan); // false most confusing gotcha!

System.out.println(Double.isNaN(nan)); // true correct check
```

```
// Infinity arithmetic

System.out.println(posInf + 1); // Infinity
System.out.println(posInf - posInf); // NaN
System.out.println(posInf * -1); // -Infinity
System.out.println(1.0 / posInf); // 0.0

// INTEGER division by zero DOES throw ArithmeticException
// int x = 5 / 0; // throws ArithmeticException: / by zero
```

IEEE 754 special values Infinity and NaN

3.4 The 0.1 + 0.2 Problem BigDecimal Solution

Binary IEEE 754 cannot represent 0.1 exactly, just as decimal cannot represent 1/3 exactly. This means floating-point calculations accumulate rounding errors. For financial calculations, BigDecimal provides exact decimal arithmetic.

```
import java.math.BigDecimal;
import java.math.RoundingMode;

// The famous floating-point precision problem
System.out.println(0.1 + 0.2); // 0.30000000000000004 NOT 0.3

// Fix 1: epsilon comparison (for physics/geometry)
double a = 0.1 + 0.2, b = 0.3;
System.out.println(Math.abs(a - b) < 1e-9); // true

// Fix 2: BigDecimal exact decimal arithmetic (for finance)
BigDecimal result = new BigDecimal("0.1").add(new BigDecimal("0.2"));
System.out.println(result); // 0.3 exact!

// TRAP: BigDecimal from double captures the imprecise double!
// new BigDecimal(0.1) // 0.10000000000000000555...
// new BigDecimal("0.1") // 0.1 ALWAYS use String constructor
```

Floating-point precision problem and BigDecimal fix

3.5 IEEE 754 Bit Layout Reference

Type	Sign Bit	Exponent Bits	Mantissa Bits	Decimal Precision
float (single)	1	8	23	~7 decimal digits
double (double)	1	11	52	~15-16 decimal digits

4 Wrapper Classes, Autoboxing & Integer Cache

4.1 Introduction Why Wrappers Exist

Java's Collections Framework and Generics work exclusively with Object references. Since primitives are not objects, each primitive type has a corresponding wrapper class in `java.lang`. The compiler automatically converts between primitive and wrapper (autoboxing/unboxing) as needed, but this convenience introduces hidden pitfalls: `NullPointerException`s from null unboxing, performance cost from boxing in tight loops, and the infamous Integer cache causing `==` comparisons to give surprising results.

4.2 Autoboxing & Unboxing What the Compiler Does

```
import java.util.List;
import java.util.ArrayList;
import java.util.Objects;

// Autoboxing: compiler inserts Integer.valueOf(i)
int i = 42;
Integer wi = i; // becomes: Integer.valueOf(42)

// Unboxing: compiler inserts wi.intValue()
int j = wi; // becomes: wi.intValue()

// Performance trap boxing in tight loop
List<Integer> list = new ArrayList<>();
for (int x = 0; x < 1_000_000; x++) {
    list.add(x); // autoboxes 1M ints -> 1M Integer objects on heap!
}

// Better: use int[] or IntStream for numeric-only computation

// NullPointerException from unboxing null
Integer nullInt = null;
int bad = nullInt; // NPE! compiler calls null.intValue()

// Guard: int safe = (nullInt != null) ? nullInt : 0;
// Or: int safe = Objects.requireNonNullElse(nullInt, 0);
```

Autoboxing and unboxing compiler transformations and pitfalls

4.3 Integer Cache The == Trap

WARNING: CRITICAL INTERVIEW TRAP: `Integer.valueOf()` caches objects for `[-128, 127]`. Values outside this range create NEW objects on every call, so `==` returns false even for logically equal values. ALWAYS use `.equals()` for Integer comparison.

```
// Cache range: -128 to 127 (JLS guaranteed minimum)
```



```

Integer a = 100; // Integer.valueOf(100) from cache

Integer b = 100; // SAME cached object

System.out.println(a == b); // true (same reference)

System.out.println(a.equals(b)); // true


Integer c = 200; // Integer.valueOf(200) new object (outside cache)

Integer d = 200; // another new object

System.out.println(c == d); // false THE TRAP

System.out.println(c.equals(d)); // true CORRECT


// Long follows the same rule

Long la = 127L, lb = 127L;

System.out.println(la == lb); // true (cached)

Long lc = 128L, ld = 128L;

System.out.println(lc == ld); // false (not cached)


// Boolean: TRUE and FALSE are singletons always cached

// Float, Double: NO cache (infinite possible values)

// -XX:AutoBoxCacheMax=N can raise the Integer upper bound (HotSpot JVM)

```

Integer cache the == trap with values inside and outside [-128,127]

4.4 Wrapper Utility Methods Reference

Wrapper	Key Methods	Notes
Integer	parseInt(s), toBinaryString(n), bitCount(n), reverse(n), compareUnsigned()	parseInt throws NumberFormatException
Long	parseLong(s), toBinaryString(n), compareUnsigned(a,b), divideUnsigned(a,b)	Unsigned arithmetic added Java 8
Double	parseDouble(s), isNaN(d), isInfinite(d), compare(a,b), sum/min/max()	compare() treats NaN==NaN (unlike ==)
Character	isDigit(c), isLetter(c), isUpperCase(c), toLowerCase(c), digit(c,radix)	Handles Unicode properties
Boolean	parseBoolean(s) only "true" (any case) is true; else false	TRUE/FALSE are singletons

5 String Immutability, Pool & Interning

5.1 Why String Is Immutable

String is declared final and its internal char array is final. Once created a String can never change. This design provides four key benefits: (1) Security: class names, file paths, and JDBC URLs passed as Strings cannot be modified after validation. (2) String pool efficiency: multiple variables can safely reference the same interned object. (3) Thread safety: immutable objects need no synchronisation. (4) hashCode caching: String lazily computes and caches its hashCode; immutability guarantees it never changes, enabling O(1) repeated hash lookups.

5.2 String Pool and the intern() Method

```
// String literals are automatically interned (stored in pool in Metaspace)

String a = "hello"; // literal -> pool

String b = "hello"; // same pool entry

System.out.println(a == b); // true same object reference!


// new String() bypasses the pool

String c = new String("hello"); // new heap object

System.out.println(c == a); // false different objects

System.out.println(c.equals(a)); // true same content


// intern() add to pool; return canonical instance

String d = c.intern(); // returns the pooled "hello"

System.out.println(d == a); // true same pooled object


// Compile-time constant folding: literals folded at compile time

String s1 = "foo" + "bar"; // "foobar" literal in pool at compile time

String s2 = "foobar";

System.out.println(s1 == s2); // true same pool entry


// Runtime concatenation creates a new heap object

String foo = "foo", bar = "bar";

String s3 = foo + bar; // runtime: new String on heap

System.out.println(s3 == s2); // false
```

String pool: literals, new String(), intern(), and constant folding

5.3 String Concatenation Performance

WARNING: Using `+` inside a loop creates a new `String` object each iteration ($O(n^2)$ total copies). Always use `StringBuilder` for multi-step or loop-driven concatenation. Java 9+ uses `invokedynamic` with `StringConcatFactory` for single-statement chains, but explicit `StringBuilder` remains necessary for loop bodies.

```
// BAD  $O(n^2)$ : creates 1000 temporary String objects

String r = "";

for (int i = 0; i < 1000; i++) r += i;


// GOOD  $O(n)$  amortized: pre-size to avoid resizing

StringBuilder sb = new StringBuilder(4096);

for (int i = 0; i < 1000; i++) sb.append(i);

String result = sb.toString(); // single allocation


// Java 15+ formatted() cleaner than String.format()

String msg = "Hello %s, age %d".formatted("Alice", 30);

// equivalent to: String.format("Hello %s, age %d", "Alice", 30)
```

String concatenation: loop trap and StringBuilder solution

6 equals() and hashCode() Contract

6.1 The Problem Silent Collection Failures

Violating the equals/hashCode contract is one of the most insidious Java bugs: objects disappear from HashMaps, Sets contain apparent duplicates, and contains() returns false for elements you just added. The bugs are silent no exception, no warning. Understanding and correctly implementing the contract is mandatory knowledge for every Java interview.

6.2 The Five Properties of equals()

Property	Definition	Violation Consequence
Reflexive	x.equals(x) must be true	Broken iterator.remove(), Set.contains()
Symmetric	x.equals(y) iff y.equals(x)	Inconsistent collection behavior
Transitive	x=y and y=z implies x=z	Breaks sorting and contains()
Consistent	Repeated calls return same result (state unchanged)	Non-deterministic HashMap behavior
Non-null	x.equals(null) must return false (not NPE)	NullPointerException in collections

hashCode contract: if a.equals(b) then a.hashCode() must equal b.hashCode(). Violating this causes HashMap.get() to return null for an existing key.

6.3 Correct Implementation Pattern

```
import java.util.Objects;

class Employee {
    private final int id;
    private final String name;
    private final String dept; // NOT included in equals (by design)

    @Override
    public boolean equals(Object o) {
        if (this == o) return true; // reflexive fast path
        if (!(o instanceof Employee e)) return false; // type + null check
        return id == e.id && Objects.equals(name, e.name); // null-safe
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name); // SAME fields as equals!
    }
}
```

```
}
```

Correct equals/hashCode using Objects.hash() and instanceof pattern (Java 16+)

6.4 Common Mistakes

```
import java.util.Set;
import java.util.HashSet;

// Mistake 1: wrong signature does NOT override Object.equals()
// public boolean equals(Employee e) { } // overloads, NOT overrides!

// Mistake 2: mutable fields in hashCode object lost in HashSet
Set<Employee> set = new HashSet<>();
Employee emp = new Employee(1, "Alice", "Eng");
set.add(emp);
emp.setName("ALICE"); // mutates hashCode -> wrong bucket!
System.out.println(set.contains(emp)); // false lost forever

// Mistake 3: override equals but NOT hashCode
// Two equal objects hash to different buckets -> HashMap.get() = null

// Best practice: use Java records for pure data carriers
record Point(int x, int y) {} // auto-generates correct equals/hashCode
```

Common equals/hashCode mistakes

TIP: Java Records (Java 16+) automatically generate equals(), hashCode(), and toString() from all record components. Use records for value objects to eliminate this entire class of bugs. Lombok @EqualsAndHashCode also generates correct implementations but requires a build-time annotation processor.

7 Pass-by-Value The Definitive Explanation

Java is ALWAYS pass-by-value. For primitives the value itself is copied. For objects, a copy of the reference (the memory address) is passed the caller's variable is never reassigned by the callee, but the object's internal state CAN be mutated through the copied reference.

```
import java.util.List;
import java.util.ArrayList;

// Case 1: Primitive copy of value
void doubleIt(int x) { x *= 2; } // only local copy changes

int n = 5;
doubleIt(n);
System.out.println(n); // still 5

// Case 2: Object copy of reference; mutation IS visible
void addElement(List<String> list) { list.add("added"); }

List<String> myList = new ArrayList<>();
addElement(myList);
System.out.println(myList.size()); // 1 mutation visible!

// Case 3: Reassignment caller's reference NOT changed
void replace(List<String> list) { list = new ArrayList<>(); } // local rebind

List<String> myList2 = new ArrayList<>(List.of("x"));
replace(myList2);
System.out.println(myList2.size()); // still 1 caller unchanged
```

Pass-by-value: primitives, object mutation, and failed reassignment

8 Bitwise Operations & Bit Manipulation Tricks

Operator	Name	Example	Result	Notes
&	AND	5 & 3	1	Both bits 1
	OR	5 3	7	Either bit 1
^	XOR	5 ^ 3	6	Different bits
~	Complement	~5	-6	Flips all bits (2's complement)
<<	Left shift	5 << 1	10	Multiply by 2
>>	Signed right shift	-8 >> 1	-4	Fills with sign bit
>>>	Unsigned right shift	-1 >>> 1	2147483647	Fills with 0 (unsigned)

```
// Practical bit tricks used in JDK source

// HashMap index: hash & (capacity-1) faster than % for power-of-2 sizes
int index = hash & (capacity - 1);

// Check if n is a power of 2
boolean isPow2 = (n > 0) && ((n & (n - 1)) == 0);

// Set / clear / toggle / test a specific bit
int setBit = val | (1 << pos); // set bit at pos
int clearBit = val & ~(1 << pos); // clear bit at pos
int toggleBit = val ^ (1 << pos); // toggle bit at pos
boolean isSet = (val & (1 << pos)) != 0; // test bit

// Count set bits (population count)
int bits = Integer.bitCount(0b1010_1111); // 6

// SAFE midpoint avoids int overflow in binary search
int mid = lo + (hi - lo) / 2; // correct
int mid2 = (lo + hi) >>> 1; // also correct (unsigned shift)

// int bad = (lo + hi) / 2; // WRONG: overflows if lo+hi > MAX_VALUE
```

Bit manipulation patterns from JDK source

Q&A Interview Questions & Answers 28 Questions

Standard questions are mid-level. [ADV] questions target senior/principal interviews.

Part 1 Primitives & Data Types

Q1. What are the 8 Java primitive types and their sizes?

byte (8-bit), short (16-bit), int (32-bit), long (64-bit), float (32-bit IEEE 754), double (64-bit IEEE 754), char (16-bit unsigned Unicode), boolean (JVM-defined, logically 1-bit). All integers use 2's complement. There is no unsigned integer type in core Java (Java 8 added unsigned helpers in Integer/Long).

Q2. What are the default values for primitive fields vs local variables?

byte/short/int/long = 0, float = 0.0f, double = 0.0d, char = '\u0000', boolean = false. Reference types = null. LOCAL variables have NO default: using them uninitialised is a compile error. Only instance and static fields receive automatic defaults.

Q3. Why can't you use primitives as generic type parameters?

Generics use type erasure and operate on reference types (Object at runtime). Primitives are not objects: they have no class metadata, no inheritance, no object header. Use wrapper classes: List not List. Project Valhalla (Java 23+) aims to eventually allow value types in generics.

Q4. What is widening conversion? Give an example where precision is lost despite widening.

Widening converts a smaller type to a larger one implicitly. Precision loss: long to float. long holds 19+ significant decimal digits but float only ~7. long l = 123_456_789_012_345L; float f = l; prints 1.2345679E14 -- digits are lost.

Q5. Explain what happens when you cast (byte) 300.

300 in binary: 0000_0001_0010_1100 (32-bit int). Narrowing to byte keeps low 8 bits: 0010_1100 = 44. The upper 24 bits are discarded. If the kept 8 bits have the high bit set, the result is negative (e.g., (byte)200 = -56).

Q6. What is numeric promotion? Why does byte a=10; byte b=20; byte c=a+b; fail to compile?

Before arithmetic on byte, short, or char, Java promotes operands to int. So a+b produces int, not byte. byte c=a+b is a compile error. Fix: byte c=(byte)(a+b). Compound operators (b+=5) include an implicit narrowing cast and DO compile.

Q7. Why does (int) 3.9 give 3 and not 4?

Narrowing float/double to integer truncates toward zero NOT rounding. (int)3.9 = 3, (int)-3.9 = -3. To round: (int)Math.round(3.9) = 4.

Q8. What is the difference between >> and >>> for a negative number?

>> is arithmetic (signed) right shift: fills vacated bits with the sign bit (1 for negative), preserving the sign. -8 >> 1 = -4. >>> is logical (unsigned) right shift: always fills with 0. -1 >>> 1 = 2147483647. Use >>> in hash functions to avoid sign propagation.

Part 2 Autoboxing, Wrappers & Integer Cache

Q9. When does autoboxing occur and what are its three most dangerous pitfalls?

Autoboxing occurs when: (1) assigning primitive to wrapper, (2) passing primitive where wrapper expected, (3) adding to a collection. Three dangers: (1) NPE: Integer x=null; int y=x calls null.intValue(). (2) == trap: Integer a=200, b=200; a==b is false (outside cache). (3) Performance: boxing in tight loops creates millions of short-lived heap objects.

Q10. Which values does the Integer cache cover? Is it configurable?

JLS guarantees caching of [-128, 127]. The upper bound can be raised with -XX:AutoBoxCacheMax=N on HotSpot (implementation detail, not JLS). Byte, Short, Long also cache [-128, 127]. Character caches [0, 127]. Boolean has TRUE/FALSE singletons. Float/Double have NO cache.

Q11. What is the output of Integer a=200; Integer b=200; System.out.println(a>b);

false. a > b triggers unboxing of both Integers to int (200 > 200 = false). Key insight: > always forces unboxing -- it cannot compare object references. Use a.compareTo(b) or Integer.compare(a,b) for safe numeric comparison.

Q12. What does Boolean.parseBoolean("TRUE") return?

true. `parseBoolean` is case-insensitive: "true", "TRUE", "True" all return `Boolean.TRUE`. Everything else ("yes", "1", "on", null, "") returns false without exception.

Part 3 String, equals/hashCode

Q13. Why is String immutable? List all four reasons.

(1) Security: class names, file paths, JDBC URLs cannot be modified after validation. (2) String pool: multiple references safely share one object. (3) Thread safety: immutable objects need no synchronisation. (4) hashCode caching: String lazily caches its hashCode; immutability guarantees it never changes.

Q14. What is String.intern() and when should you use it?

`intern()` checks the pool; if an equal String is there it returns that reference, else adds it and returns it. Use case: deduplicating large numbers of equal strings parsed from CSV. Caution: pool lives in Metaspace; interning unbounded strings causes memory pressure.

Q15. What is the equals/hashCode contract and what breaks if you violate it?

Contract: if `a.equals(b)` then `a.hashCode()` must equal `b.hashCode()`. If you override `equals` but not `hashCode`, equal objects may hash to different buckets. `HashMap.get()` returns null for your key. `HashSet.contains()` fails to find your element.

Q16. What are the five properties of equals()?

Reflexive: `x.equals(x)=true`. Symmetric: `x.equals(y)=y.equals(x)`. Transitive: `x=y` and `y=z` implies `x=z`. Consistent: same result across calls if state unchanged. Non-null: `x.equals(null)` must return false, not throw NPE.

Q17. Why should you always use .equals() instead of == for String comparison?

`==` compares object references (memory addresses), not String content. Two String objects with identical content may be different heap objects (especially after `new String(...)`, runtime concatenation, or deserialization). `.equals()` compares character-by-character content as intended.

Part 4 IEEE 754, Overflow, Pass-by-Value

Q18. What does 0.1 + 0.2 == 0.3 evaluate to?

false. 0.1 and 0.2 cannot be represented exactly in binary IEEE 754. Their sum is 0.30000000000000004. Use `Math.abs(a-b) < 1e-9` for comparison, or `BigDecimal` for exact decimal arithmetic.

Q19. What is the result of Integer.MAX_VALUE + 1?

`Integer.MIN_VALUE` (-2147483648). Java integer arithmetic silently wraps -- no exception. Use `Math.addExact(Integer.MAX_VALUE, 1)` to get `ArithmeticException: integer overflow`.

Q20. Is NaN == NaN true or false? How do you test for NaN?

false. NaN is never equal to anything, including itself -- this is the IEEE 754 specification. Use `Double.isNaN(x)` to test for NaN. `Double.compare(a,b)` treats NaN as equal to itself for sorting, which is why `Comparable` and `==` can disagree for doubles.

Q21. What is the result of 1.0/0.0 vs 0/0 in Java?

`1.0/0.0 = Double.POSITIVE_INFINITY` (no exception -- IEEE 754 float rule). `0.0/0.0 = Double.NaN`. But `0/0` (integer) throws `ArithmeticException: / by zero`. Floating-point division by zero produces special IEEE 754 values; integer division throws.

Q22. Is Java pass-by-value or pass-by-reference? Explain precisely.

Always pass-by-value. For primitives the value is copied. For objects, the VALUE of the reference (memory address) is copied. The callee can mutate the object through the copied reference (visible to caller), but reassigning the parameter only changes the local copy. Java has no pass-by-reference at all.

Q23. Can a method modify a primitive argument visible to the caller?

No. To allow a method to modify a value visible to the caller: (1) Return the new value. (2) Wrap in an array: `int[] ref = {5}; method(ref);`. (3) Use `AtomicInteger`. (4) Use a mutable holder object. Java does not have out/ref parameters like C#.

Part 5 Advanced Questions [ADV]

Q24. [ADV] How are primitives vs objects stored differently in JVM memory?

Primitives in local variables are stored directly as raw bit patterns in the stack frame -- no object header, no indirection. As instance fields they are stored inline within the object layout. Boxed wrappers (`Integer`, `Long`) are full heap objects with a 12-16 byte object header plus the value field. A raw int field = 4 bytes. An `Integer` reference+object = ~20 bytes. Primitives are

4-5x more compact.

Q25. [ADV] What is the output of: `System.out.println('a' + 'b');`

195. Char literals are promoted to int before arithmetic. 'a'=97, 'b'=98, sum=195. NOT 'ab'. To concatenate: `System.out.println("" + 'a' + 'b') = "ab"`. This is a frequent coding interview trap.

Q26. [ADV] What happens with: `char c='a'; c+=1;` vs `char c2=c+1;`

`c+=1` compiles fine -- compound operators include an implicit (char) cast. Result: `c='b'`. `char c2=c+1` is a COMPILE ERROR: `c+1` promotes to int, and int cannot be assigned to char without explicit cast. This asymmetry between `+=` and `=` is a subtle Java quirk.

Q27. [ADV] How does HashMap's internal `hash()` function improve on raw `hashCode()`?

HashMap applies: `hash = key.hashCode() ^ (h >>> 16)`. This XORs the high 16 bits with the low 16 bits, spreading high-bit entropy into the low bits. With small capacity (e.g. 16), only the low bits matter for indexing (`hash & (cap-1)`). Without spreading, hashCodes differing only in high bits would all collide in the same bucket.

Q28. [ADV] How would you safely compute the midpoint of two ints for binary search?

WRONG: `mid=(lo+hi)/2` overflows if `lo+hi > Integer.MAX_VALUE`. CORRECT: `mid=lo+(hi-lo)/2` subtraction is safe since `hi>=lo`. ALTERNATIVE: `mid=(lo+hi)>>>1` unsigned right shift avoids signed overflow. This was a real bug in Java's `Arrays.binarySearch` for 20+ years; fixed in 2006.

EX Exercises, Debug Challenges & MCQ

Part A Coding Challenges

A1. Write `safeAdd(int a, int b)` that returns `OptionalLong` containing the sum if it fits in `int`, empty otherwise. No `try/catch`, no `Math.addExact`.

A2. Implement `int reverseBytes(int n)` using bit operations only. Example: `0x12345678 -> 0x78563412`.

A3. Implement `equals()` and `hashCode()` for a 3D Point class (`double x, y, z`) that handles NaN coordinates: two Points with NaN in the same slot should NOT be equal (matching IEEE 754 double behavior).

A4. Write `parseIntSafe(String s, int defaultValue)` that returns `defaultValue` for null, empty, or non-numeric input without using exceptions for flow control.

Part B Debug & Fix

```
// B1 Integer Cache Trap

Integer score1 = 127, score2 = 127;

if (score1 == score2) { reward(); } // works by accident (cached)

Integer score3 = 200, score4 = 200;

if (score3 == score4) { reward(); } // BUG: never rewards (not cached)

// FIX: if (score3.equals(score4)) { reward(); }
```

B1: Fix the Integer cache trap

```
// B2 Overflow in Loop Bound

int count = Integer.MAX_VALUE;

for (int i = 0; i <= count; i++) { // INFINITE LOOP!

    process(i);

}

// i++ at MAX_VALUE wraps to MIN_VALUE; MIN_VALUE <= MAX_VALUE always

// FIX: for (long i = 0; i <= (long)count; i++) { ... }
```

B2: Fix the integer overflow infinite loop

```
// B3 NaN Comparison Bug

double speed = computeSpeed();

if (speed > 0) { accelerate(); } // NOT called when speed is NaN!

// NaN > 0 is false, NaN <= 0 is also false NaN fails ALL comparisons

// FIX: if (!Double.isNaN(speed) && speed > 0) { accelerate(); }
```

B3: Fix NaN comparison silent failure

```
import java.util.Map;
```

```
import java.util.HashMap;

// B4 Mutable Key in HashMap

class Range { int start, end; /* equals/hashCode on start,end */ }

Map<Range, String> map = new HashMap<>();

Range r = new Range(1, 10);

map.put(r, "hello");

r.start = 99; // mutate after insertion!

System.out.println(map.get(r)); // null object is 'lost'

// FIX: use record Range(int start, int end) {} immutable by default
```

B4: Fix mutable map key bug

Part C MCQ: Predict the Output

#	Code	Answer	Why
M C1	byte b=(byte)200; println(b);	-56	200=0xC8=1100_1000 in 8 bits -> signed = -56
M C2	println(0.1f == 0.1);	false	0.1f widened to double; bit patterns differ
M C3	char c='Z'; c+=1; println(c);	[	'Z'=90; +=1 -> 91; (char)91 = '['
M C4	Integer x=null; println(x==null?0:x);	0	x==null true; returns 0; no unboxing of null
M C5	println('a'+ 'b');	195	char arithmetic promotes to int: 97+98=195
M C6	println(Integer.bitCount(255));	8	255=0xFF=1111_1111; 8 bits set

REF Quick Reference Cheat Sheet

Topic	Key Points to Remember
8 Primitives	byte(8), short(16), int(32), long(64), float(32 IEEE), double(64 IEEE), char(16 unsigned), boolean(1*)
Default values	Numeric=0, char=\u0000, boolean=false, reference=null. Local vars MUST be initialised explicitly
Widening order	byte->short->int->long->float->double; char->int. long->float may lose precision!
Narrowing cast	Keeps low-order bits: (byte)300=44; truncates toward zero: (int)3.9=3
Numeric promotion	byte/short/char + any -> int. Mixed with long->long, float->float, double->double
Compound ops	b+=5 includes implicit (byte) cast. b=b+5 is compile error if b is byte/char
Integer overflow	Silently wraps. MAX_VALUE+1=MIN_VALUE. Use Math.addExact() for checked arithmetic
Integer cache	[-128,127] cached by Byte/Short/Integer/Long; [0,127] for Character; Boolean singletons
== on wrappers	Compares references -> use .equals() always. > < >= <= always unbox (NPE risk on null)
NaN rules	NaN != NaN (always false). Use Double.isNaN(). Double.compare() treats NaN==NaN
0.0/0.0 vs 0/0	Float/double -> NaN (no exception). Integer -> ArithmeticException: / by zero
Float precision	float~7 sig digits, double~15. Never use == on calculated floats. Epsilon or BigDecimal
String pool	Literals interned; new String() not. intern() adds to pool. Pool in Metaspace (Java 8+)
equals contract	Reflexive, Symmetric, Transitive, Consistent, Non-null. Always override hashCode with equals!
Pass-by-value	Always. Object: copy of reference (can mutate; can't reassign caller's var). No out/ref params
Bit ops	& ^ ~ << >> >>>. >>> fills with 0 (unsigned). (n&(n-1))==0 tests power-of-2
Midpoint bug	lo+(hi-lo)/2 or (lo+hi)>>>1 NOT (lo+hi)/2 which overflows
BigDecimal	Use for money. Construct from String: new BigDecimal("0.1"). Use RoundingMode.HALF_UP